DQL Commands

V P Jayachitra

The SELECT statement is the SQL command for data retrieval from relational databases.

- Part of DQL (Data Query Language)
- Read-only operation - does not modify database state
- Non-transactional - does not require COMMIT or ROLLBACK

Select statement

- Retrieve all or specific columns from tables
- Filter rows using predicates
- Sort result sets
- Perform calculations and aggregations
- Join multiple tables • Nest queries (subqueries)

Syntax

```
SELECT [DISTINCT | ALL] select_list
FROM table_reference
[WHERE search_condition]
[GROUP BY group_by_expression]
[HAVING search_condition]
[ORDER BY order_expression [ASC | DESC]];
```

Logical Query Processing Order:

1 FROM: Identify source tables
2 WHERE: Apply row-level filters
3 GROUP BY: Group rows by criteria
4 HAVING: Filter grouped results
5 SELECT: Project columns/expressions
6 DISTINCT: Eliminate duplicates
7 ORDER BY: Sort final result set

S Syntactic Writing Order
ELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY

COMPANY Database Schema

EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address,
Sex, Salary, Super_ssn, Dno)

Primary Key: Ssn

Foreign Keys: Super_ssn REFERENCES EMPLOYEE(Ssn)
Dno REFERENCES DEPARTMENT(Dnumber)

Constraints:
- Fname, Lname: NOT NULL
- Sex: CHECK (Sex IN ('M', 'F'))
- Salary: CHECK (Salary > 0)

COMPANY Database Schema

DEPARTMENT (Dname, Dnumber, Mgr_ssn, Mgr_start_date)

Primary Key: Dnumber

Unique Key: Dname

Foreign Key: Mgr_ssn REFERENCES EMPLOYEE(Ssn)

Constraints:

- Dname: NOT NULL, UNIQUE

COMPANY Database Schema

PROJECT (Pname, Pnumber, Plocation, Dnum)

Primary Key: Pnumber

Unique Key: Pname

Foreign Key: Dnum REFERENCES DEPARTMENT(Dnumber)

•

Constraints:
• Pname: NOT NULL, UNIQUE

SELECTING ALL ATTRIBUTES

Syntax:

```
SELECT *
FROM table_name;
```

- Retrieve all employee tuples
- SELECT * FROM EMPLOYEE

SELECTING SPECIFIC ATTRIBUTES

```
SELECT column1, column2, column3, ...
FROM table_name;
```

- Query 1: retrieve employee names and salary

SELECT Fname, Lname, Salary

FROM EMPLOYEE

Renaming Attributes - AS Clause

- ATTRIBUTE ALIASING MECHANISM
- Rename attributes in query result for clarity and compatibility with application requirements

```
SELECT column_name AS alias_name
FROM table_name;

Alternative Syntax (implicit):
SELECT column_name alias_name
FROM table_name;

(quoted identifiers):
SELECT column_name AS "Alias With Spaces"
FROM table_name;
```

Alias

Query 1: Explicit aliasing with AS keyword (RECOMMENDED)

```
SELECT Fname AS First_Name,
    Lname AS Last_Name,
    Salary AS Monthly_Compensation
FROM EMPLOYEE;
```

Query 2: Implicit aliasing (without AS)

```
SELECT Fname First_Name,
    Lname Last_Name,
    Salary Monthly_Compensation
FROM EMPLOYEE;
```

Alias

Query 3: Quoted identifiers for multi-word aliases

```
SELECT Fname AS "First Name",
    Lname AS "Last Name",
    Salary AS "Monthly Salary"
FROM EMPLOYEE;
```

Query 4: Aliasing computed expressions

```
SELECT Fname, Lname,
    Salary * 12 AS Annual_Salary,
    Salary * 12 * 0.20 AS Annual_Tax
FROM EMPLOYEE;
```

String Concatenation Operat

- CONCATENATION IN SQL (|| OPERATOR)
- The concatenation operator combines character strings and column values into a single string expression
- ANSI SQL Standard: || (double pipe)
- Alternative (non-standard): CONCAT(str1, str2, ...)

Syntax:

```
SELECT column1 || column2 || 'literal' AS alias
FROM table_name;
```

Query 1: Simple concatenation (no separator)

```
SELECT Fname || Lname AS Name
FROM EMPLOYEE;
```

Result: JohnSmith, FranklinWong, AliciaZelaya

Query 2: Concatenation with space separator

```
SELECT Fname || ' ' || Lname AS Full_Name
FROM EMPLOYEE;
```

Result: John Smith, Franklin Wong, Alicia Zelaya

Query 3: Three-part concatenation with middle initial

```
SELECT Fname || ' ' || Minit || '. ' || Lname
    AS Complete_Name
FROM EMPLOYEE;
```

Result: John B. Smith, Franklin T. Wong

Query 4: Concatenation with literals and expressions

```
SELECT 'Employee: ' || Fname || ' ' || Lname ||
    ' earns $' || Salary || ' monthly'
    AS Employment_Summary
FROM EMPLOYEE;
```

Result: Employee: John Smith earns $30000 monthly

Query 5: Department information formatting

```
SELECT 'Department ' || Dnumber || ': ' || Dname ||
    ' (Manager: ' || Mgr_ssn || ')'
    AS Dept_Info
FROM DEPARTMENT;
```

NULL Behavior (Critical):

- If ANY operand is NULL, entire expression evaluates to NULL
- Example: 'John' || NULL || 'Smith' → NULL
- Solution: Use NVL() or COALESCE()

```
SELECT Fname || ' ' || NVL(Minit, 'X') || '. ' || Lname
```

DISTINCT - Duplicate Elimination

- DUPLICATE ELIMINATION OPERATOR

```
Syntax:
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

- Important: DISTINCT applies to entire SELECT list, not individual columns

Query 1: Find unique department assignments

```
SELECT DISTINCT Dno
FROM EMPLOYEE;
```

Without DISTINCT: {5, 5, 4, 4, 5, 5, 4, 1} (8 values)
With DISTINCT: {1, 4, 5} (3 unique values)

Query 2: Unique salary levels in organization

```
SELECT DISTINCT Salary
FROM EMPLOYEE;
```

Result: {25000, 30000, 38000, 40000, 43000, 55000}
6 unique salary levels from 8 employees

Query 3: Unique combinations (tuple uniqueness)

```
SELECT DISTINCT Dno, Sex
FROM EMPLOYEE;
```

Result: {(1,M), (4,F), (4,M), (5,F), (5,M)}
5 unique combinations of department and gender

Query 4: Count distinct values

```
SELECT COUNT(DISTINCT Dno) AS Num_Departments
FROM EMPLOYEE;
```

Result: 3

DISTINCT vs ALL

- DISTINCT -Eliminates duplicate tuples
- ALL (default) -Retains all tuples

WHERE Clause - Selection Predicate

- ROW-LEVEL FILTERING WITH WHERE CLAUSE
- The WHERE clause implements selection operation, filtering tuples based on specified predicates.

Syntax:

```
SELECT column_list
FROM table_name
WHERE predicate;
```

Query 1: Selection by department (equality predicate)

```
SELECT Fname, Lname, Dno
FROM EMPLOYEE
WHERE Dno = 5;
```

Result: {John Smith, Franklin Wong, Ramesh Narayan,
Joyce English} - 4 tuples from Research dept

Query 2: Selection by salary threshold (comparison)

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
WHERE Salary > 40000;
```

Result: {Jennifer Wallace (43000), James Borg (55000)}

Query 3: Selection by attribute value

```
SELECT Fname, Lname, Sex
FROM EMPLOYEE
WHERE S ex= 'F';
```

Result: {Alicia Zelaya, Jennifer Wallace, Joyce English}

Query 4: Project location filter

```
SELECT Pname, Plocation
FROM PROJECT
WHERE Plocation = 'Stafford';
```

Result: {Computerize, Newbenefits}

Query 5: Date-based selection

```
SELECT Fname, Lname, Bdate
FROM EMPLOYEE
WHERE Bdate < DATE '1960-01-01';
```

Execution Semantics:

1. FROM clause identifies source relation(s)
2. WHERE clause evaluates predicate for each tuple
3. Tuples evaluating to TRUE are included
4. SELECT clause projects specified attributes

Critical Restrictions:

- WHERE clause CANNOT reference column aliases
- WHERE clause CANNOT contain aggregate functions
- Use HAVING for aggregate-based filtering

Comparison Operators

- RELATIONAL COMPARISON OPERATORS
- SQL provides standard comparison operators for predicate evaluation in WHERE and HAVING clauses.

```
 =       Equal to
<> or !=  Not equal to
<       Less than
 >       Greater than
<=       Less than or equal to
 >=      Greater than or equal to
```

Query 4: Less than or equal

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
WHERE Salary <= 30000;
```

Result: {John(30000), Alicia(25000),
Joyce(25000), Ahmad(25000)}

String Pattern Matching

- CHARACTER STRING COMPARISONS
- Comparison Semantics:
- Character-by-character comparison
- Case-sensitive by default
- Based on collation sequence

Collation Rules:
- Uppercase letters: 'A' < 'B' < ... < 'Z'
- Lowercase letters: 'a' < 'b' < ... < 'z'
- ASCII: uppercase < lowercase ('A' < 'a')
- Spaces: 'Smith' ≠ 'Smith ' (trailing space)
- Prefix: 'John' < 'Johnson' (shorter < longer)

Case-Insensitive Comparison (using functions):

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE UPPER(Fname) = 'JOHN';
```

Result: {John Smith} - Matches regardless of case

Query 1: Exact string match (equality)

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Lname = 'Smith';
```

Result: {John Smith} - Exact match required

Query 2: String inequality

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Sex != 'M';
```

Equivalent: WHERE Sex <> 'M'
Result: All female employees

Query 3: Lexicographic comparison (greater than)

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Lname > 'Smith';
```

Result: {Jennifer Wallace, Franklin Wong, Alicia Zelaya}
(Alphabetically after 'Smith')

Query 5: Case-sensitive matching

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Fname = 'JOHN';
```

Date and Temporal Comparisons

- TEMPORAL DATA TYPE COMPARISONS
- Oracle DATE data type stores both date and time components
- (century, year, month, day, hour, minute, second).

Syntax Forms:
ANSI Standard: DATE 'YYYY-MM-DD'
Oracle Function: TO_DATE('string', 'format_mask')
Current Date: SYSDATE (system function)

Query 1: Employees born before specific date

```
SELECT Fname, Lname, Bdate
FROM EMPLOYEE
WHERE Bdate < DATE '1960-01-01';
```

Result: {James Borg (1937-11-10), Franklin Wong (1955-12-08)}

Query 2: Employees born after date

```
SELECT Fname, Lname, Bdate
FROM EMPLOYEE
WHERE Bdate > DATE '1955-12-31';
```

Result: 6 employees born in 1960s

Query 3: Exact date match

```
SELECT Fname, Lname, Bdate
FROM EMPLOYEE
WHERE Bdate = DATE '1955-12-08';
```

Result: {Franklin Wong}

Query 4: Using TO_DATE with format mask

```
SELECT Fname, Lname, Bdate
FROM EMPLOYEE
WHERE Bdate < TO_DATE('01-JAN-1960', 'DD-MON-YYYY');
```

Query 5: Management start date filter

```
SELECT Dname, Mgr_start_date
FROM DEPARTMENT
WHERE Mgr_start_date >= DATE '1985-01-01';
```

Result: {Research (1988-05-22), Administration (1995-01-01)}

Query 6: Age calculation using SYSDATE

```
SELECT Fname, Lname,
    FLOOR(MONTHS_BETWEEN(SYSDATE, Bdate)/12) AS Age
FROM EMPLOYEE
WHERE Bdate < SYSDATE - (365 * 65);
```

Result: Employees over 65 years old

Logical Operators

- SQL implements Boolean logic for combining multiple predicates using AND, OR, and NOT operators
- Operator Precedence (high to low):

1. NOT
2. AND
3. OR

Use parentheses () to override precedence

Query 1: Conjunction (AND) - multiple conditions

```
SELECT Fname, Lname, Salary, Dno
FROM EMPLOYEE
WHERE Salary > 30000 AND Dno = 5;
```

Logical: σ Salary>30000 $\wedge$ Dno=5 (EMPLOYEE)

Result: {Franklin Wong(40000), Ramesh Narayan(38000)}

Query 2: Disjunction (OR) - alternative conditions

```
SELECT Fname, Lname, Dno
FROM EMPLOYEE
WHERE Dno = 4 OR Dno = 5;
```

Logical: σ Dno=4 $\vee$ Dno=5 (EMPLOYEE)

Result: 7 employees (all except James Borg)

Query 3: Negation (NOT)

```
SELECT Fname, Lname, Sex
FROM EMPLOYEE
WHERE NOT Sex = 'M';
```

BETWEEN Operator - Range Predicate

- BETWEEN operator tests whether value lies within specified range (inclusive of boundary values).

Syntax:

```
WHERE column BETWEEN lower_value AND upper_value
WHERE column NOT BETWEEN lower_value AND upper_value
```

Query 1: Salary range (numeric)

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
WHERE Salary BETWEEN 30000 AND 40000;
```

Equivalent: WHERE Salary >= 30000 AND Salary <= 40000
Result: {John(30000), Ramesh(38000), Franklin(40000)}

Query 2: NOT BETWEEN (exclusion)

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
WHERE Salary NOT BETWEEN 30000 AND 40000;
```

Equivalent: WHERE Salary < 30000 OR Salary > 40000
Result: {Alicia(25000), Joyce(25000), Ahmad(25000),
Jennifer(43000), James(55000)}

Query 3: Date range (temporal)

```
SELECT Fname, Lname, Bdate
FROM EMPLOYEE
WHERE Bdate BETWEEN DATE '1950-01-01'
        AND DATE '1960-12-31';
```

Result: Employees born in 1950s decade

Query 4: Department number range

```
SELECT Pname, Dnum
FROM PROJECT
WHERE Dnum BETWEEN 1 AND 5;
```

Result: All projects (departments 1, 4, 5)

Query 5: Lexicographic range (strings)

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Lname BETWEEN 'A' AND 'M';
```

Result: Surnames alphabetically from A through M
{James Borg, Joyce English, Ahmad Jabbar}

note:

- BETWEEN is INCLUSIVE (includes both boundaries)
- Lower bound must precede upper bound
- Boundary order: BETWEEN 1 AND 5 (NOT 5 AND 1)
- Works with numeric, string, and date types

IN Operator - Set Membership

- IN operator tests whether value matches any element in specified value list.

Syntax:
WHERE column IN (value1, value2, value3, ...)
WHERE column NOT IN (value1, value2, value3, ...)

Advantages over Multiple OR:
- More concise syntax
- Improved readability
- Better query optimizer recognition
- Supports subquery expressions

Query 1: Department membership test

```
SELECT Fname, Lname, Dno
FROM EMPLOYEE
WHERE Dno IN (4, 5);
```

Equivalent: WHERE Dno = 4 OR Dno = 5
Result: 7 employees from Administration and Research depts

Query 2: Exclusion with NOT IN

```
SELECT Fname, Lname, Dno
FROM EMPLOYEE
WHERE Dno NOT IN (1, 4);
```

Equivalent: WHERE Dno != 1 AND Dno != 4
Result: 4 employees (only Research dept)

Query 3: String value set

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Lname IN ('Smith', 'Wong', 'Zelaya');
```

Result: {John Smith, Franklin Wong, Alicia Zelaya}

LIKE Operator - Pattern Matching

- STRING PATTERN MATCHING WITH LIKE
- % Matches zero or more characters
- _ Matches exactly one character

Syntax:
WHERE column LIKE 'pattern'
WHERE column NOT LIKE 'pattern'

Case Sensitivity: LIKE is case-sensitive
WHERE Fname LIKE 'j%' → matches nothing
WHERE UPPER(Fname) LIKE 'J%' → case-insensitive

Query 1: Prefix matching (starts with)

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Fname LIKE 'J%';
```

Pattern: J followed by any characters
Result: {John, Jennifer, Joyce, James}

Query 2: Suffix matching (ends with)

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Lname LIKE '%son';
```

Pattern: Any characters followed by 'son'
Result: None in sample data (would match Johnson, Wilson)

Query 3: Substring matching (contains)

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Fname LIKE '%ar%';
```

Pattern: 'ar' anywhere in string
Result: {Ramesh Narayan} ('ar' in Narayan)

Query 4: Fixed-length pattern

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Fname LIKE '____';
```

Pattern: Exactly 4 characters (4 underscores)
Result: {John} (only 4-character first name)

Query 5: Positional character matching

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Fname LIKE '_a%';
```

Pattern: Second character is 'a'
Result: {Ramesh, James}

Query 6: Negation with NOT LIKE

```
SELECT Dname
FROM DEPARTMENT
WHERE Dname NOT LIKE 'R%';
```

Result: {Administration, Headquarters}

NULL Value Hand

- NULL represents absence of value - unknown or inapplicable data.
- NULL is NOT a value; it's a marker for missing information.

Syntax:
WHERE column IS NULL
WHERE column IS NOT NULL

Query 1: Employees without supervisor

```
SELECT Fname, Lname, Super_ssn
FROM EMPLOYEE
WHERE Super_ssn IS NULL;
```

Result: {James Borg} (CEO has no supervisor)

Query 2: Employees with supervisor

```
SELECT Fname, Lname, Super_ssn
FROM EMPLOYEE
WHERE Super_ssn IS NOT NULL;
```

Result: 7 employees (all except CEO)

Query 3: Check for missing middle initial

```
SELECT Fname, Minit, Lname
FROM EMPLOYEE
WHERE Minit IS NULL;
```

Result: Employees without middle initial (if any)

Query 4: Filter non-NULL values

```
SELECT Fname, Minit, Lname
FROM EMPLOYEE
WHERE Minit IS NOT NULL;
```

COMMON ERROR :

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Super_ssn = NULL;     ← WRONG! Returns EMPTY SET

WHERE Super_ssn != NULL;    ← WRONG! Returns EMPTY SET
```

CORRECT APPROACH:

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Super_ssn IS NULL;    ← CORRECT!

WHERE Super_ssn IS NOT NULL; ← CORRECT!
```